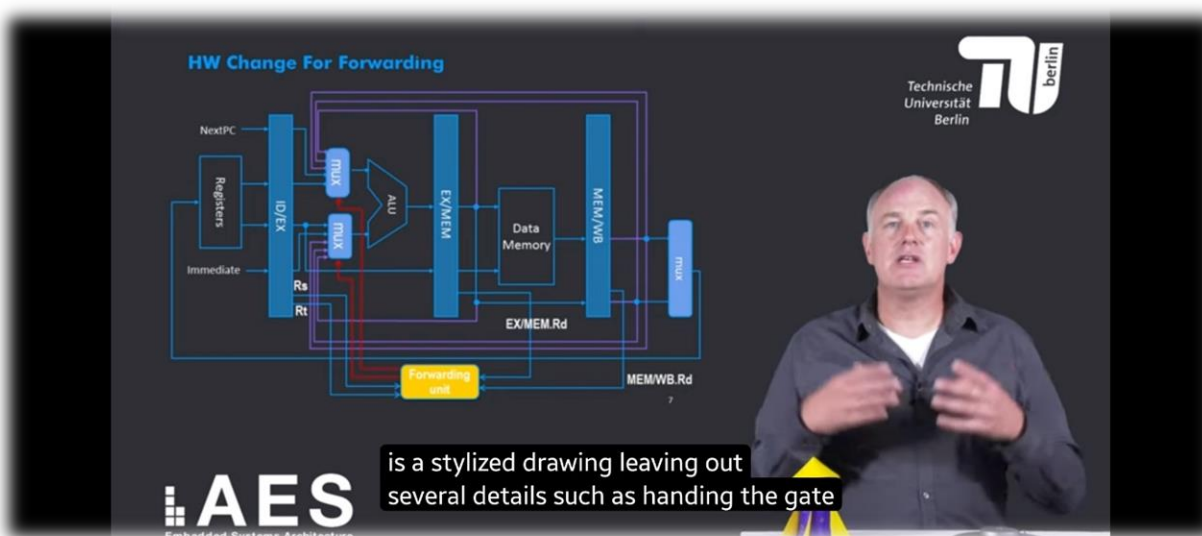


طراحی و پیاده سازی Forwarding

در بخش اول پروژه، پردازنده‌ی تک‌سیکلی پایه به یک پایپ‌لاین ۵ مرحله‌ای تبدیل شد. در چنین پایپ‌لاینی، در هر لحظه چند دستور بطور هم‌زمان در مراحل مختلف در حال پردازش هستند. این هم‌زمانی، یک مشکل بنیادین ایجاد می‌کند: اگر دستوری به مقدار رجیستری نیاز داشته باشد که هنوز توسط دستور قبلی نوشته نشده، مقدار خوانده‌شده از رجیستر فایل نادرست خواهد بود. به این مشکل، **Data Hazard** گفته می‌شود.

راه حل :



به جای Stall کردن پایپ‌لاین تا نوشتن مقدار در رجیستر فایل، مقدار صحیح مستقیماً از میانه‌ی پایپ‌لاین گرفته می‌شود و قبل از ورود به ALU جایگزین مقدار خوانده‌شده از رجیستر فایل می‌شود.

منابع فوروارد در این طراحی:

- **EX/MEM** : نتیجه‌ی ALU دستور یک مرحله جلوتر
- **MEM/WB** : مقدار نهایی دستور دو مرحله جلوتر، شامل نتیجه‌ی ALU یا داده‌ی خوانده‌شده از حافظه

تذکر : اگر هر دو معتبر باشند، **EX/MEM** اولویت دارد چون تازه تر است.

تغییرات ایجاد شده :

1. EXC_ID (اصلاح شده)

علاوه بر مقادیر RS1 و RS2 ، شماره این رجیسترها نیز تا مرحله Execute منتقل می شود تا امکان مقایسه با مقصد دستورات قبلی وجود داشته باشد.

2. EXMEM_Fwd_Pipe (جدید)

اطلاعات زیر را نگهداری می کند:

- خروجی ALU
- شماره رجیستر مقصد (Rd)
- سیگنال RegWrite

3. Forwarding_Unit (جدید)

در هر سیکل:

- RS1
- RS2

را با رجیستر مقصد موجود در مراحل:

- EX/MEM
- MEM/WB

مقایسه می کند ، در صورت برقرار بودن شرایط، سیگنال Forward فعال می شود.

4. چهار تا Multiplexer (جدید)

این مالتی پلکسرها: مقدار خوانده شده از Register File یا مقدار Forward شده را به درستی انتخاب کرده و به ALU ، Branch Unit ، مسیر Store ارسال می کنند.

معادلات منطقی Forwarding Unit :

برای هر یک از RS1/RS2 مشابه است

ForwardA_EXMEM =(EXMEM.Rd == RS1)

AND EXMEM.RegWrite

AND (EXMEM.Rd $\neq$ 0)

ForwardA_MEMWB =(MEMWB.Rd == RS1)

AND MEMWB.RegWrite

AND (MEMWB.Rd $\neq$ 0)

AND NOT(ForwardA_EXMEM)

برای RS2 نیز دقیقاً همین روابط برقرار است.

علت شرط $Rd \neq 0$

در معماری RISC-V رجیستر **x0** همیشه صفر است و هیچ‌گاه نوشته نمی‌شود.

اگر این شرط وجود نداشته باشد، دستوراتی مانند NOP نیز به اشتباه در حال نوشتن تشخیص داده می‌شوند.

البته یک سوال مهم هنوز پابرجا است که آیا همه حالت‌های وابستگی با این روش برطرف خواهند شد؟

خیر .

Data hazard به خوبی برطرف میشه ولی این روش نمیتواند مشکل **Load-Use Hazard** را حل کند.

به عبارتی تنها Hazard باقی مانده، Load-Use Hazard است که به دلیل محدودیت زمانی ذاتی نیازمند یک سیکل Stall توسط Hazard Detection Unit است .

زمانی که دستور نویسنده‌ی رجیستر از نوع **Load** باشد و بلافاصله پس از آن، دستور دیگری از همان رجیستر استفاده کند، حتی سریع‌ترین مسیر **Forwarding** (از مسیر EX/MEM) نیز قادر به رفع مخاطره نخواهد بود. علت این موضوع، **محدودیت زمانی ذاتی** در پایپ‌لاین است و نه کمبود مسیرهای فوروارد یا سیم‌کشی.

سیکل	I1: lw x1, 0(x2)	I2: add x3, x1, x4
۳	EX (تنها آدرس $x2 + offset$ محاسبه می‌شود).	ID
۴	MEM (داده در همین سیکل از حافظه خوانده می‌شود).	EX (در همین لحظه به مقدار x1 نیاز دارد).
۵	WB	MEM

همان‌طور که در جدول بالا مشاهده می‌شود، در سیکل ۴ داده‌ی موردنیاز هنوز در حال خوانده‌شدن از حافظه است و در هیچ‌یک از ثبات‌های پایپ‌لاین وجود ندارد تا بتوان آن را **Forward** کرد. در این لحظه، ثبات EX/MEM تنها آدرس محاسبه‌شده‌ی حافظه را در اختیار دارد و هنوز داده‌ی بارگذاری‌شده در آن قرار نگرفته است.

بنابراین، مشکل ناشی از نبود مسیر **Forwarding** نیست، بلکه مقدار موردنیاز در آن سیکل هنوز به‌صورت فیزیکی تولید نشده است و در نتیجه هیچ مسیری برای انتقال آن وجود ندارد.